

Testbench:

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity alu1bit_tb is  
end alu1bit_tb;
```

architecture behavior of alu1bit_tb is

```
    component alu1bit  
    port (  
        a      : in std_logic;  
        b      : in std_logic;  
        carryin : in std_logic;  
        blnvert : in std_logic;  
        op      : in std_logic_vector(1 downto 0);  
        z       : out std_logic;  
        carryout : out std_logic;  
        zero    : out std_logic;  
        less    : out std_logic;  
        overflow : out std_logic  
    );  
end component;
```

```
    signal ta, tb, tcarryin, tblnvert : std_logic;  
    signal top : std_logic_vector(1 downto 0);  
    signal tz, tcarryout, tzero, tless, toverflow : std_logic;
```

begin

```
    uut: alu1bit port map (  
        a      => ta,  
        b      => tb,  
        carryin => tcarryin,  
        blnvert => tblnvert,  
        op      => top,  
        z       => tz,  
        carryout => tcarryout,  
        zero    => tzero,  
        less    => tless,  
        overflow => toverflow  
    );
```

tb_proc: process

begin

```
    -- Test ADD (op = "00", blnvert = '0')  
    top <= "00"; tblnvert <= '0';
```

ta <= '0'; tb <= '0'; tcarryin <= '0'; wait for 30 ns;
assert tz = '0' and tcarryout = '0' report "ADD 0+0+0 failed";

ta <= '0'; tb <= '0'; tcarryin <= '1'; wait for 30 ns;
assert tz = '1' and tcarryout = '0' report "ADD 0+0+1 failed";

ta <= '0'; tb <= '1'; tcarryin <= '0'; wait for 30 ns;
assert tz = '1' and tcarryout = '0' report "ADD 0+1+0 failed";

ta <= '0'; tb <= '1'; tcarryin <= '1'; wait for 30 ns;
assert tz = '0' and tcarryout = '1' report "ADD 0+1+1 failed";

ta <= '1'; tb <= '0'; tcarryin <= '0'; wait for 30 ns;
assert tz = '1' and tcarryout = '0' report "ADD 1+0+0 failed";

ta <= '1'; tb <= '0'; tcarryin <= '1'; wait for 30 ns;
assert tz = '0' and tcarryout = '1' report "ADD 1+0+1 failed";

ta <= '1'; tb <= '1'; tcarryin <= '0'; wait for 30 ns;
assert tz = '0' and tcarryout = '1' report "ADD 1+1+0 failed";

ta <= '1'; tb <= '1'; tcarryin <= '1'; wait for 30 ns;
assert tz = '1' and tcarryout = '1' report "ADD 1+1+1 failed";

-- Test SUBTRACT (op = "00", blnvert = '1', carryin = '1')
top <= "00"; tblnvert <= '1'; tcarryin <= '1';

ta <= '0'; tb <= '0'; wait for 30 ns;
assert tz = '1' and tcarryout = '1' report "SUB 0-0 failed";

ta <= '0'; tb <= '1'; wait for 30 ns;
assert tz = '0' and tcarryout = '0' report "SUB 0-1 failed";

ta <= '1'; tb <= '0'; wait for 30 ns;
assert tz = '0' and tcarryout = '1' report "SUB 1-0 failed";

ta <= '1'; tb <= '1'; wait for 30 ns;
assert tz = '1' and tcarryout = '1' report "SUB 1-1 failed";

-- Test AND (op = "01", blnvert irrelevant)
top <= "01"; tblnvert <= '0'; tcarryin <= '0';

ta <= '0'; tb <= '0'; wait for 30 ns;
assert tz = '0' report "AND 0&0 failed";

ta <= '0'; tb <= '1'; wait for 30 ns;
assert tz = '0' report "AND 0&1 failed";

```

    ta <= '1'; tb <= '0'; wait for 30 ns;
    assert tz = '0' report "AND 1&0 failed";

    ta <= '1'; tb <= '1'; wait for 30 ns;
    assert tz = '1' report "AND 1&1 failed";

    -- Test OR (op = "10", blnvert irrelevant)
    top <= "10"; tblnvert <= '0'; tcarryin <= '0';

    ta <= '0'; tb <= '0'; wait for 30 ns;
    assert tz = '0' report "OR 0I0 failed";

    ta <= '0'; tb <= '1'; wait for 30 ns;
    assert tz = '1' report "OR 0I1 failed";

    ta <= '1'; tb <= '0'; wait for 30 ns;
    assert tz = '1' report "OR 1I0 failed";

    ta <= '1'; tb <= '1'; wait for 30 ns;
    assert tz = '1' report "OR 1I1 failed";

    report "Testbench finished successfully";
    wait;
end process;
end behavior;

```

AMU:

```

library ieee;
use ieee.std_logic_1164.all;

```

```

entity alu1bit is
port (
    a      : in std_logic;
    b      : in std_logic;
    carryin : in std_logic;
    blnvert : in std_logic;
    op      : in std_logic_vector(1 downto 0);
    z       : out std_logic;
    carryout : out std_logic;
    zero    : out std_logic;
    less    : out std_logic;
    overflow : out std_logic
);

```

```
end alu1bit;
```

architecture behavior of alu1bit is

```
    signal b_actual : std_logic;  
    signal sum_result : std_logic;  
    signal and_result : std_logic;  
    signal or_result : std_logic;  
    signal operation_result : std_logic;  
    signal internal_carryout : std_logic;
```

```
begin
```

```
    b_actual <= not b when bInvert = '1' else b;
```

```
    sum_result <= a xor b_actual xor carryin;  
    internal_carryout <= (a and b_actual) or (carryin and (a xor b_actual));
```

```
    and_result <= a and b;  
    or_result <= a or b;
```

```
    with op select operation_result <=  
        sum_result when "00", -- ADD/SUB  
        and_result when "01", -- AND  
        or_result  when "10", -- OR  
        '0'        when others;
```

```
    z <= operation_result;  
    carryout <= internal_carryout when op = "00" else '0';
```

```
    zero <= not operation_result;  
    less <= a and (not b) when op = "00" and bInvert = '1' else '0';  
    overflow <= carryin xor internal_carryout when op = "00" else '0'; -  
end behavior;
```